

CorreIA — CRM Imobiliário Inteligente

Este é o repositório do projeto **CorreIA**, um CRM Imobiliário Inteligente desenvolvido com **Next.js (App Router)**, **TypeScript**, **Tailwind CSS** e animações fluidas com **GSAP**. A arquitetura de dados e autenticação utiliza **Supabase** como banco de dados PostgreSQL e storage de mídias, integrado com **autenticação JWT própria** persistida via cookies **httpOnly**.

Principais Módulos e Recursos Desenvolvidos

1. Autenticação JWT Própria (Pessoa 1 — Breno)

- Login e cadastro próprios, sem dependência de serviços externos para sessão.
- Tokens JWT assinados no servidor e salvos em cookies **httpOnly; Secure; SameSite=Strict**, garantindo imunidade contra ataques XSS.
- Criptografia de senhas com **bcrypt** no servidor.
- Proteção de rotas privativas server-side via **src/proxy.ts** (Next.js 16).

2. CRM / Módulo de Leads & Kanban (Giuseppe)

- Gestão completa de Leads (criação, edição, exclusão e detalhamento).
- Quadro Kanban interativo com 5 colunas representando as etapas do funil de vendas (**Novo** → **Em atendimento** → **Visita agendada** → **Proposta** → **Fechado**).
- Mapeamento robusto entre as propriedades em formato camelCase no cliente e snake_case no banco de dados.

3. Catálogo de Imóveis (Gabriel Brandão)

- CRUD completo de imóveis (Casa, Apartamento, Terreno, Comercial).
- Integração com o **Supabase Storage** para upload de fotos do imóvel direto do formulário de criação/edição.
- Controle de status de disponibilidade do imóvel (**Disponível**, **Vendido**, **Alugado**).

4. Visual Premium & Landing Page (Gustavo Gurgel)

- Identidade visual moderna baseada em Dark Mode e Glassmorphism.
- Landing page com Seção Hero dinâmica e um **Cinematic Footer** interativo equipado com **GSAP** e **ScrollTrigger**, incluindo efeitos magnéticos e animações guiadas pelo rolamento da página.

Estrutura de Pastas Atualizada

```
src/
├── app/
│   ├── api/
│   │   ├── auth/
│   │   │   ├── login/route.ts    # POST – valida credenciais e emite JWT
│   │   │   └── register/route.ts # POST – cria conta com hash bcrypt + emite
```

2 / 5

```

├── contexts/
│   └── AuthContext.tsx          # Estado global de autenticação
├── hooks/
│   ├── useAuth.ts              # Hook de atalho para AuthContext
│   ├── useImoveis.ts           # Hook de integração com Supabase para imóveis
│   └── useLeads.ts              # Hook para comunicação com API Routes de
leads
├── lib/
│   ├── api-auth.ts             # Auxiliar de leitura JWT em API Routes
│   ├── jwt.ts                  # signToken() e verifyToken()
│   └── leads-mapper.ts         # Tradutor de camelCase <-> snake_case para
leads
├── leads
│   ├── leads.ts                # Requisições HTTP para API Routes de leads
│   └── supabase.ts             # Clients públicos e privados do Supabase
├── types/
│   └── lead.ts                 # Interfaces de tipos e constantes do funil
└── proxy.ts                    # Middleware de roteamento e validação server-
side

```

🔑 Configuração das Variáveis de Ambiente

Crie um arquivo `.env.local` na raiz do projeto:

```

# JWT – NUNCA adicionar NEXT_PUBLIC_ aqui (server-only)
JWT_SECRET=<gerar com: openssl rand -base64 64>

# Supabase – service_role para API Routes (server-only, sem NEXT_PUBLIC_)
SUPABASE_SERVICE_ROLE_KEY=<pegar em: Supabase Dashboard → Project Settings → API
Keys → service_role>

# Supabase – público para queries client-side
NEXT_PUBLIC_SUPABASE_URL=https://SEU_PROJECT_ID.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=SUA_ANON_KEY_DO_SUPABASE

```

🗄️ Modelagem do Banco de Dados (SQL do Supabase)

Execute os comandos a seguir no painel **SQL Editor** do Supabase para inicializar as tabelas necessárias:

```

-- 1. Tabela de Perfis de Usuários
CREATE TABLE perfis (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nome_completo text NOT NULL,
  email       text UNIQUE NOT NULL,
  cpf         text,
  creci       text,
  senha_hash  text NOT NULL,
  role        text DEFAULT 'corretor', -- 'corretor' | 'lead' |

```

```
'admin_corretora' | 'super_admin'
criado_em      timestamp with time zone DEFAULT timezone('utc', now()) NOT NULL
);

ALTER TABLE perfis DISABLE ROW LEVEL SECURITY;

-- 2. Tabela de Leads
CREATE TABLE leads (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nome          text NOT NULL,
  telefone      text NOT NULL,
  email         text NOT NULL,
  faixa_orcamento text NOT NULL,
  tipo_imovel   text NOT NULL,
  etapa         text NOT NULL DEFAULT 'novo',
  corretor_id   uuid REFERENCES perfis(id) ON DELETE SET NULL,
  criado_em     timestampz DEFAULT timezone('utc', now()) NOT NULL,
  atualizado_em timestampz DEFAULT timezone('utc', now()) NOT NULL
);

ALTER TABLE leads DISABLE ROW LEVEL SECURITY;

-- 3. Tabela de Imóveis
CREATE TABLE imoveis (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tipo          text NOT NULL, -- 'casa' | 'apartamento' | 'terreno' | 'comercial'
  endereco      text NOT NULL,
  bairro        text NOT NULL,
  valor         numeric NOT NULL,
  metragem      numeric,
  quartos       integer,
  vagas         integer,
  status        text NOT NULL DEFAULT 'disponivel', -- 'disponivel' | 'vendido' |
  'alugado'
  imagem_url    text,
  criado_em     timestamp with time zone DEFAULT timezone('utc', now()) NOT NULL
);

ALTER TABLE imoveis DISABLE ROW LEVEL SECURITY;
```

Configuração do Supabase Storage

Para o upload de fotos dos imóveis funcionar:

1. Vá até o menu **Storage** no console do Supabase.
2. Crie um novo Bucket de armazenamento.
3. Nomeie o bucket exatamente como `imoveis_imagens`.
4. Certifique-se de configurar o bucket como **Public** para permitir que as URLs de visualização sejam carregadas publicamente nos cards.

Controle de Rotas

A segurança de rotas privadas é baseada em `src/proxy.ts`. Caso queira proteger uma nova página, lembre-se de adicionar o padrão dela na configuração de `matcher`:

```
export const config = {
  matcher: ['/', '/dashboard/:path*', '/imoveis/:path*', '/leads/:path*',
    '/funil/:path*', '/outra-pagina/:path*'],
};
```

Resgatar dados de Autenticação no Client

```
import { useAuth } from '@/hooks/useAuth';

export default function Exemplo() {
  const { profile, loading } = useAuth();

  if (loading) return <div>Carregando...</div>;
  return <div>Nome: {profile?.nome_completo} ({profile?.role})</div>;
}
```

Utilizando Serviços no Backend (API Routes)

Para realizar queries que necessitam ultrapassar restrições de permissões ou ler dados sigilosos no backend, use o client com `service_role`:

```
import { createServerSupabaseClient } from '@/lib/supabase';

const supabaseServer = createServerSupabaseClient();
const { data } = await supabaseServer.from('perfis').select('*');
```

Comandos do Projeto

- `pnpm install` — Instala as dependências do projeto.
- `pnpm dev` — Inicia o servidor de desenvolvimento local.
- `pnpm build` — Compila a aplicação para produção e roda as validações estáticas/TypeScript.
- `pnpm start` — Inicializa o servidor Next.js construído para produção.